

Week 08: Interpreters with explicit errors

Functional Programming in Lean

Learning goals

- ▶ Model expression syntax as a datatype
- ▶ Write a compositional evaluator
- ▶ Locate the precise point where an error arises

Central idea

An interpreter follows the syntax tree. Each recursive result is available inside do.

Lean example

```
def eval : Expr → Except String Int
| .lit n => .ok n
| .add x y => do
  let a ← eval x
  let b ← eval y
  pure (a + b)
```

What to notice

- ▶ The AST separates syntax from meaning
- ▶ An error can stop an evaluation branch
- ▶ Adding syntax requires a new evaluator case

Live coding

1. Add the division constructor
2. Reject a zero divisor
3. Trace the evaluation of a nested expression

Pause and predict

Which subexpression is evaluated first in `add x y`?

Exercise connection

Open `exercises/Week08.md` after the lecture.

The complete example is `Course/Week08.lean`; the fill-in version is `Templates/Week08.lean`.